

Relativity Toolkit v1.13.1

```
7  (* ===== *)
8  (* RELATIVITY TOOLKIT ENGINE (Script Mode) *)
9  (* Version: 1.13.1 (Bug fix: jk1234 identified as Extended format) *)
10 (* ----- *)
11 (* TODO: complete the removal of hard-coded  $\Gamma$ . Currently, only torsionRules, *)
12 (* CD, and SetConnection know about RelativityConnection. There are several *)
13 (* places in MakeBoxes and other display functions that hard-code  $\Gamma$ . *)
14 (* ===== *)
15
16 Echo[RelativityToolkitVersion = "1.13.1", "Relativity Toolkit version :"];
17
18 (* CONFIGURATION: The Connection Symbol *)
19 (* Default is  $\Gamma$ ; change it to A, C, etc., for Electrodynamics, Yang-Mills, etc. *)
20
21 Echo[RelativityConnection =  $\Gamma$ , "Relativity Connection Symbol :"];
22
```

Clean Slate

```
29 (* ===== *)
30 (* 1. CLEAN SLATE ----- *)
31 (* ===== *)
32
33 Unprotect[MakeBoxes];
34
35 (* Unset explicit Toolkit symbols *)
36 Quiet[MakeBoxes[Partials[__], StandardForm] =.];
37 Quiet[MakeBoxes[CD[__], StandardForm] =.];
38 Quiet[MakeBoxes[ $\delta$ [__], StandardForm] =.];
39 Quiet[MakeBoxes[ $\Gamma$ [__], StandardForm] =.];
40
41 (* Clear the generic U/D rules by checking ONLY the Pattern (LHS), not the RHS *)
42 (* This removes rules that MATCH on U/D, but saves rules that just USE U/D internally *)
43 DownValues[MakeBoxes] = Select[DownValues[MakeBoxes],
44   FreeQ[First[#],  $\mathcal{U}$ ] && FreeQ[First[#],  $\mathcal{D}$ ] &
45 ];
46
```

```
Protect[MakeBoxes];
```

```
Module[{symbols = Symbol /@ {
  "CalculateRicciComponent",
  "CalculateRiemannComponent",
  "CalculateGammaComponent",
  "CanonicalizeIndices",
  "CanonicalizeTerm",
  "CD",
  "checkFormalIdentity",
  "ChristoffelsFromMetric",
  "ChristoffelsFromMetricIndexer",
  "Contract",
  "ContractAll",
  "ContractIndexed1",
  "ContractIndexed2",
  "ContractIndexedAll",
  "contractValence",
  "CreateExtendedFormal",
  "differentiationRules",
  "downQ",
  "EvaluateUDPartials",
  "ExpandDerivatives",
  "ExtractCoefficient",
  "formalCharCodeQ",
  "FormalSymbolQ",
  "FormalSymbolExtendedQ",
  "g", (* reserved for metric tensors (will be relaxed later) *)
  "GradRaised",
  "GradSquared",
  "ruleLeviCivita",
  "MakeIndexer",
  "MatrixToUDRules",
  "metricDifferentiationRules",
  "metricRules",
  "noValence",
  "Partials",
  "RelativityConnection",
  "RelativityToolkitVersion",
  "robustTransformRules",
  "ScalarLaplacian",
  "SetConnection",
  "showCodes",
  "SymbolInspector",
```

```

"TensorForm",
"torsionRules",
"upQ",
"valence",
(* TODO: Protect all these symbols ! *)
"x", (* all coordinate functions are named x *)
"D", (* essential syntax *)
"U", (* essential syntax *)
"I", (* reserved for GR *)
"δ" (* reserved for the Kronecker symbol *)
}},
ClearAll[Sequence[symbols]];
ClearAll[
  λ, κ, ρ, σ, μ, ν, τ, η, χ, ψ,
  x, p, g, u, A, B, T,
  P, Q, S]];

```

Formal Symbols

formalCharCodeQ

showCodes

checkFormalIdentity

FormalSymbolQ

FormalSymbolExtendedQ

CreateExtendedFormal

```

123 (* FORMAL SYMBOLS *)
124
125 (* from analysis of Wolfram documentation *)
126 formalCharCodeQ[code_Integer] := (
127     (63488 ≤ code ≤ 63556) || (*Latin and primary math*)
128     (63558 ≤ code ≤ 63564) || (*Remaining math symbols after arrow*)
129     (63572 ≤ code ≤ 63596) || (*Lowercase Greek*)
130     (63604 ≤ code ≤ 63605) || (*Mathematical variants*)
131     (63608 ≤ code ≤ 63609) || (*Greek variants*)
132     (63613 ≤ code ≤ 63622) || (code === 63626) || (*Final Greek variants & oddities*)
133     (1024000 ≤ code ≤ 1024051));
134
135 showCodes[] := Module[{codes = Range[63488, 63615]},
136     Grid[Partition[MapIndexed[Column[{
137         Item[Style[Symbol@FromCharacterCode[#1], 20, Bold],
138             Background → If[formalCharCodeQ[#1],
139                 Darker[StandardGreen],
140                 Darker[StandardRed]]],
141         Style[#2[[1]] + 63487, 10, GrayLevel[0.90]]],
142         Alignment → Center] &, codes], 16],
143     Frame → All,
144     ItemSize → {2.5, 2.5},
145     Background → {None, None, {}}]];
146
147
148 SetAttributes[checkFormalIdentity, HoldFirst];
149 checkFormalIdentity[s:(_Symbol|_String)] :=
150     Module[{name, codes},
151         name = Which[
152             Head[Unevaluated[s]] === Symbol, SymbolName[Unevaluated[s]],
153             (* if input is a literal string, not a symbol *)
154             StringQ[Unevaluated[s]], s,
155             True, Return[$Failed]];
156         codes = ToCharacterCode[name];
157         Which[
158             (Length[codes] == 1 && formalCharCodeQ[First[codes]]), "Pure",
159             (Length[codes] > 0 &&
160                 formalCharCodeQ[First[codes]] &&
161                 !AnyTrue[Rest[codes], formalCharCodeQ]), "Extended",
162             True, "Neither"
163         ];
164     checkFormalIdentity[___] := $Failed;

```

```

167 SetAttributes[{FormalSymbolQ},HoldAll];
168 FormalSymbolQ[s_]:= "Pure"===checkFormalIdentity[s];

171 SetAttributes[{FormalSymbolExtendedQ},HoldAll];
172 FormalSymbolExtendedQ[s_]:=
173     checkFormalIdentity[s]=== "Extended"&&MemberQ[Attributes[s],Protected];
174
175
176 CreateExtendedFormal::invalidStructure="The name '`1`' is invalid. Extended formal
177 Module[{valid},
178     valid[codes_]:=Length[codes]>0&&
179         formalCharCodeQ[First[codes]]&&
180         !AnyTrue[Rest[codes],formalCharCodeQ];
181     CreateExtendedFormal[name_String]:=
182         If[valid[ToCharacterCode[name]],
183             With[{s=Symbol[name]},
184                 Protect[s];s],
185             (Message[CreateExtendedFormal::invalidStructure,name];
186                 Return[$Failed])];
187     CreateExtendedFormal[sym_Symbol]:=
188         CreateExtendedFormal[SymbolName[sym]]; ];

```

Valence and Type Checker

upQ

downQ

contractValence

noValence

valence

```

203 (* ===== *)
204 (* 2. VALENCE LOGIC (The Type Checker) *)
205 (* ===== *)
206
207 upQ[x_] := upQ[valence[x]];
208 downQ[x_] := downQ[valence[x]];
209 upQ[{{__}, {}]} := True; upQ[___] := False;
210 downQ[{{}}, {__}] := True; downQ[___] := False;
211
212 contractValence[{u_List, d_List}] :=
213     With[{common = Intersection[u, d]},
214         {DeleteElements[u, common],
215         DeleteElements[d, common]};

```

```

noValence := {{}, {}]; (* := (set delayed) means fresh instance each time *)

valence[x_Symbol] := noValence;
valence[_?NumericQ] := noValence;
valence[] := Null;

valence[_[ $\mathcal{U}[\alpha]$ ]] := {{ $\alpha$ }, {}];
valence[_[ $\mathcal{D}[\alpha]$ ]] := {{}, { $\alpha$ }};

valence[g[ $\mathcal{D}[\mu]$ ],  $\mathcal{D}[\nu]$ ]] := {{}, { $\mu$ ,  $\nu$ }};
valence[g[ $\mathcal{U}[\mu]$ ],  $\mathcal{U}[\nu]$ ]] := {{ $\mu$ ,  $\nu$ }, {}];
valence[ $\delta[\mathcal{U}[\mu]$ ],  $\mathcal{D}[\nu]$ ]] := {{ $\mu$ }, { $\nu$ }};

valence[Partials[num_, den_]] :=
  Module[{vn = valence[num], vd = valence[den], vdFlipped},
    vdFlipped = {vd[[2]], vd[[1]]};
    {Join[vn[[1]], vdFlipped[[1]], Join[vn[[2]], vdFlipped[[2]]]};

valence[CD[num_, den_]] := valence[Partials[num, den]];

(* TODO: Opportunity for replacement by RelativityConnection *)
valence[I[ $\mathcal{U}[a]$ ],  $\mathcal{D}[b]$ ],  $\mathcal{D}[c]$ ]] := {{a}, {b, c}};

valence[Derivative[1][f_][arg_]] :=
  Module[{u, d},
    {u, d} = valence[arg];
    {d, u}];

valence[Derivative[_][_][f_] /; (valence[f] != noValence)] :=
  valence[f];
203
204
205
206
207
208
209
210
211
212
213
214
215

```

```

valence[Power[b_, _]] := valence[b];

valence[h_[a_, b_, rest___]] :=
  Join[valence[h[a]], valence[h[b, rest]], 2];

valence[___] := noValence;

```

Display Rules

MakeBoxes[CD, ...]

MakeBoxes[Partials, ...]

MakeBoxes[Γ , ...]

MakeBoxes[δ , ...]

MakeBoxes[h_[idcs___], ...]

```

281 (* ===== *)
282 (* 3. DISPLAY RULES *)
283 (* ===== *)
284
285 Unprotect[MakeBoxes];
286
287 (* Semicolon Notation for CD *)
288 MakeBoxes[CD[num_, den_], StandardForm] :=
289   If[MatchQ[den, x[_]],
290     Replace[den,
291       x[_[idx_]]  $\Rightarrow$ 
292         SubscriptBox[MakeBoxes[num, StandardForm],
293           RowBox[{";", MakeBoxes[idx, StandardForm]}]],
294     RowBox[{"CD", "[" , MakeBoxes[num, StandardForm], ",",
295       MakeBoxes[den, StandardForm], "]" }]];
296
297 (* PARTIAL DERIVATIVES (The Comma Operator) *)
298
299 MakeBoxes[Partials[num_, den_], StandardForm] :=
300   If[MatchQ[num, Partials[_ , _]],
301
302     (* CASE 1: Nested Derivative  $\rightarrow$  Fraction *)
303     Replace[num,
304       Partials[top_, bot1_]  $\Rightarrow$ 
305       FractionBox[
306         RowBox[{SuperscriptBox[" $\partial$ ", "2"], MakeBoxes[top, StandardForm]}],
307         RowBox[{RowBox[{" $\partial$ ", MakeBoxes[den, StandardForm]}],

```

```

RowBox[{" $\partial$ ", MakeBoxes[bot1, StandardForm]}]]]],

(* CASE 2 & 3 *)
If[MatchQ[den, x[u[_]]] && ! MatchQ[num, x[u[_]]],

(* CASE 2: Comma Notation *)
Replace[den,
  x[u[idx_]]  $\Rightarrow$ 
  SubscriptBox[
    (* Wrap in parentheses if Product/Sum/Power *)
    If[MemberQ[{Times, Plus, Power}, Head[num]],
      RowBox[{"(", MakeBoxes[num, StandardForm], ")"}],
      MakeBoxes[num, StandardForm]],

    (* Comma and Index *)
    RowBox[{"", MakeBoxes[idx, StandardForm]}]]],

281 (* CASE 3: Default  $\rightarrow$  Fraction *)
282 FractionBox[
283   RowBox[{" $\partial$ ", MakeBoxes[num, StandardForm]}],
284   RowBox[{" $\partial$ ", MakeBoxes[den, StandardForm]}]] ];
285
286 (* TODO: replace with RelativityConnection *)
287 MakeBoxes[I[u[_], d[d1_], d[d2_]], StandardForm] :=
288   SubsuperscriptBox["I",
289     RowBox[{MakeBoxes[d1, StandardForm], MakeBoxes[d2, StandardForm]}],
290     MakeBoxes[u, StandardForm]];
291
292 MakeBoxes[ $\delta$ [u[up_], d[down_]], StandardForm] :=
293   SubsuperscriptBox[" $\delta$ ", MakeBoxes[down, StandardForm],
294     MakeBoxes[up, StandardForm]];
295
296 MakeBoxes[ $\delta$ [d[down_], u[up_]], StandardForm] :=
297   SubsuperscriptBox[" $\delta$ ", MakeBoxes[down, StandardForm],
298     MakeBoxes[up, StandardForm]];
299
300 MakeBoxes[h_[indices_], StandardForm] /;
301   (h !=  $\delta$ ) &&
302   (h != Partial) &&
303   (h != CD) &&
304   (h != I) &&
305   (* Hold for safe inspection *)
306   MatchQ[Hold[indices], Hold[(u | d | u | d) ...]] :=
307   Module[{formattedScripts},

```

```

(* Handle the replacement safely, though usually
   if we matched the pattern, the args are just symbols/indices anyway. *)
formattedScripts = {indices} /. {u[i_] =>
  SuperscriptBox["",
    MakeBoxes[i, StandardForm]], d[i_] =>
  SubscriptBox["", MakeBoxes[i, StandardForm]]};
RowBox[Prepend[formattedScripts, MakeBoxes[h, StandardForm]]];

Protect[MakeBoxes];

```

Tensor-Algebra Rules

CanonicalizeTerm

CanonicalizeIndices

robustTransformRules

TensorForm

ExtractCoefficient

```

376 (* ===== *)
377 (* 4. ALGEBRAIC SIMPLIFICATION *)
378 (* ===== *)
379
380 CanonicalizeTerm[term_] :=
381   Module[{indices, counts, dummies, replacementRules},
382     indices = Cases[term, (u | d)[i_] => i, Infinity];
383     counts = Tally[indices];
384     dummies = Select[counts, Last[#] == 2 &][[All, 1]];
385     replacementRules = MapIndexed[
386       #1 -> CreateExtendedFormal["i" <> ToString[First[#2]]] &,
387       dummies];
388     term /. replacementRules];
389
390 CanonicalizeIndices[expr_Plus] :=
391   Total[CanonicalizeIndices /@ List @@ expr];
392
393 CanonicalizeIndices[expr_Equal] :=
394   Equal @@ (CanonicalizeIndices /@ List @@ expr);
395
396 CanonicalizeIndices[expr_] := CanonicalizeTerm[expr];
397
398 robustTransformRules = {
399   A_[u[primed_]] =>

```

```

With[{fresh = CreateExtendedFormal@Unique["μ"]},
  Partials[x[u[primed]], x[u[fresh]]] * A[u[fresh]],
p[_[D[primed_]]] ==>
  With[{fresh = CreateExtendedFormal@Unique["ν"]},
    Partials[x[u[fresh]], x[u[primed]]] * p[D[fresh]]];

(* DYNAMIC TENSOR FORM *)
TensorForm[expr_] :=
  Module[{canonExpr, formalIndices,
    usedSymbols, greekPool, availableGreek, indexMap},

    (* 1. Canonicalize first *)
    canonExpr = CanonicalizeIndices[expr];

    (* 3. Identify ALL Formal Indices *)
    formalIndices = Sort @ DeleteDuplicates @ Cases[canonExpr,
      s_Symbol /; (FormalSymbolQ[s] || FormalSymbolExtendedQ[s]), Infinity];

    (* 4a. Identify symbols ALREADY in the expression *)
    376 usedSymbols = DeleteDuplicates @ Cases[canonExpr, _Symbol, Infinity];
    377
    (* 4b. Define preferred Greek pool *)
    378 greekPool = {λ, κ, ρ, σ, μ, ν, τ, η, χ, ψ, α, β, γ};
    379
    380
    (* 5. Filter the pool: Remove symbols that are already used *)
    381 availableGreek = DeleteElements[greekPool, usedSymbols];
    382
    383
    (* 6. Map the ordered, found formals to the available Greek letters *)
    384 indexMap = Thread[formalIndices -> Take[availableGreek, UpTo[Length[formalIndices]]];
    385
    386
    (* 7. Apply *)
    387 HoldForm[Evaluate[canonExpr /. indexMap]] ];
    388
    389
    (* EXTRACT COEFFICIENT FIELD (Quotient-Theorem Tool) *)
    390 ExtractCoefficient[expr_, field_Symbol] :=
    391 Module[{processTerm, expanded, termList},
    392   expanded = Expand[expr];
    393   termList = If[Head[expanded] === Plus,
    394     List @@ expanded,
    395     {expanded}];
    396
    397
    processTerm[term_] := Module[{canonTerm, dummyIndex, targetIdx, normalizedTerm},
    398   (* 1. Canonicalize locally *)
    399

```

```

canonTerm = CanonicalizeIndices[term];

(* 2. Find index attached to the field *)
dummyIndex = Cases[canonTerm, field[_[idx_]] :> idx, Infinity];
If[dummyIndex === {}, Return[0]];

(* 3. NORMALIZE: Swap the field's index to a Reserved Symbol (Formals) *)
(* preventing collision with internal dummies (i1, i2) in other terms *)
targetIdx = First[dummyIndex];
normalizedTerm = canonTerm /. targetIdx -> Symbol["ṡ"];

(* 4. Extract coefficient of A^S *)
Coefficient[normalizedTerm, field[ṡ[Symbol["ṡ"]]]] +
Coefficient[normalizedTerm, field[ḍ[Symbol["ṡ"]]]];

Total[processTerm /@ termList];

```

Application-Specific Rules

metricRules

torsionRules

ruleLeviCivita

metricDifferentiationRules

```

473 (* ===== *)
474 (* 5. APPLICATION-SPECIFIC RULES *)
475 (* User must explicitly apply these when wanted *)
476 (* ===== *)
477
478 metricRules = {
479   g[ḍ[mu_], ḍ[nu_]]*vec_[ṡ[nu_]] :> vec[ḍ[mu]],
480   vec_[ṡ[nu_]]*g[ḍ[mu_], ḍ[nu_]] :> vec[ḍ[mu]],
481
482   g[ḍ[nu_], ḍ[mu_]]*vec_[ṡ[nu_]] :> vec[ḍ[mu]],
483   vec_[ṡ[nu_]]*g[ḍ[nu_], ḍ[mu_]] :> vec[ḍ[mu]],
484
485   g[ṡ[mu_], ṡ[nu_]]*covec_[ḍ[nu_]] :> covec[ṡ[mu]],
486   covec_[ḍ[nu_]]*g[ṡ[mu_], ṡ[nu_]] :> covec[ṡ[mu]],
487
488   g[ṡ[nu_], ṡ[mu_]]*covec_[ḍ[nu_]] :> covec[ṡ[mu]],
489   covec_[ḍ[nu_]]*g[ṡ[nu_], ṡ[mu_]] :> covec[ṡ[mu]],
490   (* TODO: incorporate symmetry rules rather than brute case enumeration *)

```

```

g[U[μ_], U[α_]]*g[D[α_], D[v_]] := δ[U[μ], D[v]],
g[U[α_], U[μ_]]*g[D[α_], D[v_]] := δ[U[μ], D[v]],

g[D[α_], D[v_]]*g[U[μ_], U[α_]] := δ[U[μ], D[v]],
g[D[v_], D[α_]]*g[U[μ_], U[α_]] := δ[U[μ], D[v]],

g[U[μ_], U[α_]]*g[D[v_], D[α_]] := δ[U[μ], D[v]],
g[U[α_], U[μ_]]*g[D[v_], D[α_]] := δ[U[μ], D[v]],

g[D[α_], D[v_]]*g[U[α_], U[μ_]] := δ[U[μ], D[v]],
g[D[v_], D[α_]]*g[U[α_], U[μ_]] := δ[U[μ], D[v]],

(* δ-Contraction Rules (For Associativity) *)

(* Rule: T^β Subscript[δ^α, β] → T^α *)
expr_ * δ[U[b_], D[a_]] /; !FreeQ[expr, D[b]] :=
  (expr /. D[b] → D[a]),
δ[U[b_], D[a_]] * expr_ /; !FreeQ[expr, D[b]] :=
  (expr /. D[b] → D[a]),

(* Rule: Subscript[T, α Subscript[δ^α, β] → Subscript[T, β] *)
expr_ * δ[U[a_], D[b_]] /; !FreeQ[expr, U[b]] :=
  (expr /. U[b] → U[a]),
δ[U[a_], D[b_]] * expr_ /; !FreeQ[expr, U[b]] :=
  (expr /. U[b] → U[a]),

473 (* Cleanup: Delta on itself *)
474 δ[U[a_], D[b_]] * δ[U[b_], D[c_]] := δ[U[a], D[c]]
475
476 };
477
478 (* Torsion-Free Geometry (General Relativity) *)
479 (* User must apply manually: expr //. torsionRules *)
480 torsionRules = {
481   RelativityConnection[u_, a_, b_] :=
482     RelativityConnection[u, Sort[{a, b}][[1]], Sort[{a, b}][[2]]
483 };
484
485 ruleLeviCivita=R[U[λ_],D[a_],D[b_]]:=
486   With[{σ = CreateExtendedFormal@Unique["σ"]},
487     1/2 g[U[λ],U[σ]]*
488     (Partials[g[D[σ],D[b]],x[U[a]]]+
489     Partials[g[D[σ],D[a]],x[U[b]]]-
490     Partials[g[D[a],D[b]],x[U[σ]]]);

```

```

metricDifferentiationRules={
  (*Metric Symmetry*)
  g[D[a_],D[b_]]/;!OrderedQ[{a,b}]:>g[D[b],D[a]],
  g[U[a_],U[b_]]/;!OrderedQ[{a,b}]:>g[U[b],U[a]],
  (*Permissive Contraction (All 4 Alignments)*)
  (*First-First*)
  g[D[s_],D[a_]]*g[U[s_],U[b_]]:>δ[U[b],D[a]],
  (*Second-Second*)
  g[D[a_],D[s_]]*g[U[b_],U[s_]]:>δ[U[b],D[a]],
  (*First-Second (Redundant but Safe)*)
  g[D[s_],D[a_]]*g[U[b_],U[s_]]:>δ[U[b],D[a]],
  (*Second-First*)
  g[D[a_],D[s_]]*g[U[s_],U[b_]]:>δ[U[b],D[a]],
  (*Delta Contractions*)
  δ[U[bound_],D[free_]]*Partials[expr_,x[U[bound_]]]:>Partials[expr,x[U[free]]],
  δ[U[bound_],D[free_]]*Partials[g[D[bound_],D[other_]],v_]:>Partials[g[D[free],D[
  δ[U[bound_],D[free_]]*Partials[g[D[other_],D[bound_]],v_]:>Partials[g[D[other],D[

```

Differentiation Engine

differentiationRules

ExpandDerivatives

Partials

```

566 (* ===== *)
567 (* 6. DIFFERENTIATION ENGINE *)
568 (* ===== *)
569
570 differentiationRules = {
571   Partials[a_ * b_, var_]  $\Rightarrow$  Partials[a, var]*b + a*Partials[b, var],
572   Partials[a_ + b_, var_]  $\Rightarrow$  Partials[a, var] + Partials[b, var],
573   Partials[Exp[arg_], var_]  $\Rightarrow$  Exp[arg] * Partials[arg, var],
574   Partials[a_^n_, var_]  $\Rightarrow$  n a^(n-1) * Partials[a, var],
575   Partials[n_, _] /; (NumericQ[n])  $\Rightarrow$  0,
576   Partials[expr_, x[u[idx_]]] /; StringContainsQ[ToString[idx], "'"]  $\Rightarrow$ 
577     With[{fresh = CreateExtendedFormal@Unique[" $\sigma$ "]},
578       Partials[x[u[fresh]], x[u[idx]]]*
579       Partials[expr, x[u[fresh]]]];
580
581 ExpandDerivatives[expr_] := expr //. differentiationRules;
582
583 (* Schwarz's Theorem (Sort Partial Derivatives) *)
584 (* If variables are out of canonical order, swap them *)
585 Partials[Partials[f_, x[u[a_]]], x[u[b_]]] /;
586   !OrderedQ[{a, b}] :=
587   Partials[Partials[f, x[u[b]]], x[u[a]]];
588

```

Covariant Derivative

SetConnection

CD

GradRaised

GradSquared

ScalarLaplacian

```

603 (* ===== *)
604 (* 7. COVARIANT DERIVATIVE *)
605 (* ===== *)
606
607 SetConnection[sym_Symbol] := (
608   RelativityConnection = sym;
609   Print["Relativity Engine: Connection set to ", sym]
610 );
611
612 (* Rule 0: Connection-Binding ("Gamma Glue") *)
613 (* DYNAMIC: Check if prod contains the CURRENT RelativityConnection symbol *)

```

```

(* treating 'Γ * A' as a single atom *)
CD[prod_Times /; !FreeQ[prod, RelativityConnection], var_] :=
  Module[{up, down, partialTerm, upCorrections, downCorrections},

    {up, down} = valence[prod];
    partialTerm = Partials[prod, var];

    (* Apply Corrections using the CURRENT Connection Symbol *)
    upCorrections = Sum[
      With[{lam = CreateExtendedFormal@Unique["λ"]},
        RelativityConnection[U[idx], D[var[[1,1]], D[lam]] * (prod /. idx → lam)
      ], {idx, up}];

    downCorrections = Sum[
      With[{lam = CreateExtendedFormal@Unique["λ"]},
        RelativityConnection[U[lam], D[idx], D[var[[1,1]]] * (prod /. idx → lam)
      ], {idx, down}];

    partialTerm + upCorrections - downCorrections ];

(* Linearity *)
CD[a_ + b_, var_] := CD[a, var] + CD[b, var];

(* Product Rule *)
CD[a_ * b_, var_] := CD[a, var] * b + a * CD[b, var];

(* The General Tensor Rule *)
CD[expr_, x[U[nu_]]] :=
  Module[{up, down, partialTerm, upCorrections, downCorrections},

    {up, down} = valence[expr];
    partialTerm = Partials[expr, x[U[nu]]];

    (* DYNAMIC: Generate corrections using RelativityConnection *)
    upCorrections = Sum[
      With[{lam = CreateExtendedFormal@Unique["λ"]},
        RelativityConnection[U[idx], D[nu], D[lam]] * (expr /. idx → lam)
      ], {idx, up}];

    downCorrections = Sum[
      With[{lam = CreateExtendedFormal@Unique["λ"]},
        RelativityConnection[U[lam], D[idx], D[nu]] * (expr /. idx → lam)
      ], {idx, down}];

```

```

partialTerm + upCorrections - downCorrections ];

(* Mixed Geometric & Gauge Derivative *)
(* Usage: CD[field, coord, coupling, gaugeField] *)
(* Returns: GeometricCD - i * coupling * gaugeField_mu * field *)
CD[field_, var_, coupling_, gaugeBoson_Symbol] :=
  CD[field, var] - I * coupling * gaugeBoson[D[var[[1,1]]] * field;

(* Gradient *)
GradRaised[func_, gInvUU_Symbol, gInvUURules_, coords_] :=
  Table[Sum[(gInvUU[u[mu],u[nu]]/. gInvUURules)D[func,nu],
    {nu,coords}],{mu,coords}];

(* Gradient Squared *)
GradSquared[func_, gInvUU_Symbol, gInvUURules_, coords_] :=
  With[{grad = MakeIndexer[GradRaised[func, gInvUU, gInvUURules, coords], coords]},
    Simplify[
      Sum[D[func, mu]*grad[mu], {mu, coords}]]];

(* Scalar Laplacian *)
ScalarLaplacian[func_, gInvUU_Symbol, gInvUURules_, sqrtDetg_, coords_] :=
  With[{grad=MakeIndexer[GradRaised[func,gInvUU,gInvUURules,coords],coords]},
    Simplify[
      Sum[D[sqrtDetg *grad[mu],mu],{mu,coords}]/sqrtDetg]];

```

Gamma, Riemann, Ricci

ChristoffelsFromMetric

ChristoffelsFromMetricIndexer

CalculateGammaComponent

CalculateRiemannComponent

CalculateRicciComponent

```

697 (* ===== *)
698 (* 8. RIEMANN CURVATURE TENSOR FROM METRIC *)
699 (* ===== *)
700
701 (* Christoffel (Γ) *)
702
703 (* Reminder: this is not a general connection, but one for torsion-free, *)
704 (* symmetric metrics, such as for relativistic gravitation. *)

```

```
(* Send in metric and its inverse so as not to recompute inverse. *)
(* The symbol for covariant gDD must not equal that for contravariant gUU. *)
```

```
ChristoffelsFromMetric[gDD_, gUU_,  $\sigma$ _,  $\mu$ _,  $\nu$ _]/;(gDD != gUU) :=
```

```
With[{ $\lambda$  = CreateExtendedFormal@Unique[" $\lambda$ "]},
  (1/2) gUU[ $\mathcal{U}[\sigma], \mathcal{U}[\lambda]$ ]  $\times$ 
  (Partials[gDD[ $\mathcal{D}[\lambda]$ ,  $\mathcal{D}[\mu]$ ],  $\mathcal{X}[\mathcal{U}[\nu]]$ ] +
   Partials[gDD[ $\mathcal{D}[\lambda]$ ,  $\mathcal{D}[\nu]$ ],  $\mathcal{X}[\mathcal{U}[\mu]]$ ] -
   Partials[gDD[ $\mathcal{D}[\mu]$ ,  $\mathcal{D}[\nu]$ ],  $\mathcal{X}[\mathcal{U}[\lambda]]$ ])];
```

```
(* A version that works from metrics made via MakeIndexer *)
```

```
ChristoffelsFromMetricIndexer[g_, gi_,  $\sigma$ _,  $\mu$ _,  $\nu$ _, coords_] :=
Sum[(1/2) gi[ $\sigma, \lambda$ ] (D[g[ $\lambda, \mu$ ],  $\nu$ ] + D[g[ $\lambda, \nu$ ],  $\mu$ ] - D[g[ $\mu, \nu$ ],  $\lambda$ ]),
  { $\lambda$ , coords}];
```

```
CalculateGammaComponent[
```

```
   $\sigma$ _,  $\mu$ _,  $\nu$ _,
  gDD_Symbol, gDDRules_,
  gInvUU_Symbol, gInvUURules_,
  coords_] :=
(ContractAll[
  ChristoffelsFromMetric[gDD, gInvUU,  $\sigma$ ,  $\mu$ ,  $\nu$ ], coords]/.
  gDDRules/.
  gInvUURules)//
EvaluateUDPartials;
```

```
(* Riemann *)
```

```
(* Send in IGet so as not to recompute it on every call. *)
```

```
(* e.g., IGet=MakeIndexer[SchwI, {t, r,  $\theta$ ,  $\phi$ }] *)
```

```
CalculateRiemannComponent[ $\mu$ Up_,  $\lambda$ Dn_,  $\nu$ Dn_,  $\rho$ Dn_, Iget_, coords_List] :=
```

```
Module[{dI1, dI2, interact1, interact2},
  (* Derivative Terms *)
  dI1 = D[Iget[ $\mu$ Up,  $\lambda$ Dn,  $\nu$ Dn],  $\rho$ Dn];
  dI2 = D[Iget[ $\mu$ Up,  $\lambda$ Dn,  $\rho$ Dn],  $\nu$ Dn];
  (* Interaction Terms: *)
  Module[{ $\kappa$ },
    interact1 = Sum[Iget[ $\kappa$ ,  $\lambda$ Dn,  $\nu$ Dn] * Iget[ $\mu$ Up,  $\kappa$ ,  $\rho$ Dn], { $\kappa$ , coords}];
    interact2 = Sum[Iget[ $\kappa$ ,  $\lambda$ Dn,  $\rho$ Dn] * Iget[ $\mu$ Up,  $\kappa$ ,  $\nu$ Dn], { $\kappa$ , coords}];
    Simplify[dI1 - dI2 + interact1 - interact2];
```

```
(* Ricci *)
```

```
(* Send in RGet so as not to recompute it each time *)
(* e.g., RGet = MakeIndexer[SchwRiemann, {t, r,  $\theta$ ,  $\phi$ }] *)
CalculateRicciComponent[ $\lambda$ _,  $\rho$ _, RGet_, coords_] :=
Module[{ $\kappa$ },
  Sum[RGet[ $\mu$ ,  $\lambda$ ,  $\mu$ ,  $\rho$ ], { $\mu$ , coords}] // Simplify];
```

Compiler

MatrixToUDRules

Contract

ContractAll

EvaluateUDPartials

MakeIndexer

ContractIndexed1

ContractIndexed2

determineArity

ContractIndexedAll

```
773 (* ===== *)
774 (* 9. COMPILER AND SUPPORT FUNCTIONS *)
775 (* ===== *)
776
777 (* MatrixToUDRules *)
778 (* Use Case: Convert a Wolfram Matrix to 2-D UD Rules *)
779 (* Input: A square, 2D matrix 'mat' of components,
780          a Symbol 'g' to affect, e.g. gDD or gUU,
781          a valence indicator,  $\mathcal{U}$  or  $\mathcal{D}$ .
782          a List of coordinate symbols, e.g., {t, r,  $\theta$ ,  $\phi$ } *)
783 (* Notes: Works only for 'homovalent' symbols like gDD or gUU.
784          Heterovalent symbols like gUD or gDU are not handled here. *)
785 (* Output: A list of rules like {gDD[ $\mathcal{D}$ [0],  $\mathcal{D}$ [0]]  $\rightarrow$  A[r], ...} *)
786
787 MatrixToUDRules[mat_, g_Symbol, UorD_ /; (UorD ===  $\mathcal{U}$  || UorD ===  $\mathcal{D}$ ), coords_List]
788 Module[{dim, rules},
789   dim = Length[mat];
790   rules = Flatten @ Table[
791     g[UorD[coords[[i]], UorD[coords[[j]]]]  $\rightarrow$  mat[[i, j]],
792     {i, 1, dim}, {j, 1, dim}];
793   (* Filter out zeros to keep the rule list short *)
794   Select[rules, (#[[2]] != 0) &];
795
796 (* Contraction *)
```

```

(* In imitation of Wolfram's Replace and ReplaceAll, we present *)
(* contraction functions that Total a list of terms with bound *)
(* indices (repeated up-down indices in a term) where the bound *)
(* indices are replaced by all the coords, in order. *)

(* Contract performs contraction on the first detected bound *)
(* index and ContractAll performs contraction on all bound indices *)

(* Contract on the first bound index (repeated up-down in a term) *)
Contract[expr_, coords_List] :=
  Module[{canonicalExpr, oneShotScanner, result, PD},

    (* Convert "Partial w.r.t Upper" (x^i) to "lower inert operator" (PD_i) *)
    (* Explicitly creates the D[i] index needed for scanning. *)
    canonicalExpr = expr /. Partial[f_, x[U[i_]]] => PD[f, D[i]];

    (* Find a bound index pair that is not a coordinate symbol. *)
    oneShotScanner[term_] := Module[{ups, downs, pairs, idx},
      ups = Cases[term, U[s_Symbol] => s, Infinity];
      downs = Cases[term, D[s_Symbol] => s, Infinity];

      (* Exclude coordinate names from contraction *)
      pairs = Complement[Intersection[ups, downs], coords];

      If[Length[pairs] == 0,
        term,
        (idx = First[pairs];
          (* Total up on the expansion over all coords *)
          Total[term /. idx -> # & /@ coords])];

    (* Apply the scanner to all terms with the same bound index *)
    With[{ec = Expand[canonicalExpr]},
      result = If[Head[ec] == Plus,
        Map[oneShotScanner, ec],
        oneShotScanner[ec]];

    (* Restore Partial from PD for downstream UD evaluation. *)
    result /. PD[f_, D[i_]] => Partial[f, x[U[i]]];

    (* ContractAll: Contract on all bound indices (repeated up-down in a term) *)
    ContractAll[expr_, coords_List] := Module[{
      canonicalExpr, recursiveScanner, result, PD},

```

```

(* PHASE A: Canonicalize Valence *)
(* Convert "Partial w.r.t Upper" ( $x^i$ ) to "lower inert operator" (PD_i) *)
(* Explicitly creates the D[i] index needed for scanning. *)
canonicalExpr = expr /. Partial[f_, x[u[i_]]]  $\Rightarrow$  PD[f, D[i]];

(* PHASE B: The Recursive Scanner *)
recursiveScanner[term_] := Module[{ups, downs, pairs, idx},
  ups = Cases[term, u[s_Symbol]  $\Rightarrow$  s, Infinity];
  downs = Cases[term, D[s_Symbol]  $\Rightarrow$  s, Infinity];

  (* Exclude coordinate names from contraction *)
  pairs = Complement[Intersection[ups, downs], coords];

  If[Length[pairs] === 0,
    term,
    (idx = First[pairs];
     (* Recurse on the expansion *)
     recursiveScanner[Total[term /. idx  $\rightarrow$  # & /@ coords]]]]];

773 (* Run Scanner on Expanded Expression *)
774 With[{ec = Expand[canonicalExpr]},
775   result = If[Head[ec] === Plus,
776     Map[recursiveScanner, ec],
777     recursiveScanner[ec]]];
778
779 (* PHASE C: Restoration *)
780 (* Convert internal PD back to Partial for UD. *)
781 result /. PD[f_, D[i_]]  $\Rightarrow$  Partial[f, x[u[i]]];
782
783
784 (* Compile UD partials into Wolfram D expressions. *)
785 EvaluateUDPartials[expr_] := expr /.
786   With[{killBogusChainRule = {(u|D)'[idx_]  $\rightarrow$  0}},
787     {
788       (* CASE: Standard UD Syntax *)
789       Partial[f_, x[u[idx_]]]  $\Rightarrow$  (D[f, idx] /. killBogusChainRule),
790
791       (* CASE: Pretty-Printed Subscript Notation (The comma) *)
792       Subscript[f_, {idx_}]  $\Rightarrow$  (D[f, idx] /. killBogusChainRule),
793
794       (* CASE: Derivatives of Numerical Partial *)
795       Partial[n_, x[(u|D)[_]]] /; NumberQ[n]  $\Rightarrow$  0,
796       Partial[(E | I |  $\pi$ ), x[(u|D)[_]]]  $\Rightarrow$  0

```

```
}};
```

```
(* Use Case: Make a function that can retrieve components from a Wolfram array
(*           that represents a tensor-like object using symbolic coordinate symbols
(* Usage (example): Iget = MakeIndexer[Itable, {t, r,  $\theta$ ,  $\phi$ });
(*           Iget[r, t, t] ~-> A'[r]/(2 B[r])
(* Input: `table` is a 1-indexed Wolfram array of any dimension containing
(*           components of a tensor-like object such as  $\Gamma$ , g, or R.
(*           `coords` is a list of coordinate symbols, such as {t, r,  $\theta$ ,  $\phi$ }.
(*           The Length of coords must match the dimension of the array (unchecked).
(* Return: a function that looks up components in the array by coord symbols.
```

```
MakeIndexer[table_List,coords_List] :=
  With[{rules=MapIndexed[(#1->#2[[1]])&,coords]}],
    table[[Sequence@@({##} //. rules)]]&;
```

```
ContractIndexed1[f1_,coords_] := Sum[f1[k],{k,coords}];
ContractIndexed2[f2_,coords_] := Sum[f2[j,k],{j,coords},{k,coords}];
```

773

774 (*1. Helper to deduce function arity*)

775 ClearAll[determineArity];

776 determineArity[f_] :=

777 Module[{lhs},

778 Which[

779 (*Explicit Function[{x,y},body]*)

780 MatchQ[f, HoldPattern[Function[{_}, _]]], Length[f[[1]]],

781 (*Explicit Function[x,body] (Single arg)*)

782 MatchQ[f, HoldPattern[Function[_Symbol, _]]], 1,

783 (*Pure Function with Slots (#1,#2)*)

784 MatchQ[f, HoldPattern[Function[_]]],

785 With[{slots=Cases[f, Slot[i_Integer]>:i, Infinity]}],

786 If[slots==={}, (*If no numbered slots, fail for ## (SlotSequence),

787 If[FreeQ[f, SlotSequence], 0, \$Failed],

788 Max[slots]]],

789 (*Symbol with DownValues (e.g. foo[a_,b_] := ...)*)

790 MatchQ[f, _Symbol]&&Length[DownValues[f]]>0,

791 Extract[

792 First[DownValues[f]],

793 {1,1},

794 Function[term, Length[Unevaluated[term]], HoldAll]],

795 (*Default: Cannot determine*)

796 True, \$Failed]];

```

(*2. The Generalized Contractor*)
ClearAll[ContractIndexedAll];
ContractIndexedAll[func_,coords_List,explicitArity:_Integer|Automatic:Automatic]:=
Module[{arity,dummies,iterators},
(*Determine how many indices to generate*)
arity=If[IntegerQ[explicitArity],
explicitArity,
determineArity[func]];(*'Automatic' case*)
(*Safety Check*)
If[!IntegerQ[arity]||arity<1,
(Message[ContractIndexedAll::noarity,func];
Return[$Failed])];
(*Generate Dummies and Sum*)
dummies=Table[Unique["idx"],arity];
iterators={#,coords}&/@dummies;
(*Apply func to dummies (f[i,j,k]) and Sum*)
Sum[func@@dummies,Evaluate[Sequence@@iterators]]];

773 (*Error Message definition*)
774 ContractIndexedAll::noarity="Could not automatically determine the arity of funct-

```

Gauge Theory

CurvatureForcesFromConnectionIndexer
AbsoluteDerivativeFromConnectionIndexer
PotentialScalarFieldFromMetricIndexer
CurvatureForcesUD
AbsoluteDerivativeUD
PotentialScalarFieldUD

```

966 CurvatureForcesFromConnectionIndexer[v_,Γ_,time_,coords_]:=
967   With[{injectTime=Flatten[{c→{c→c[time],c'→c'[time]}]/@coords}],
968     Table[
969       ContractIndexedAll[
970         {v,λ}→(Γ[μ,v,λ]/.injectTime)v[v]×v[λ],
971         coords],
972     {μ,coords}]]];

```

```

975 AbsoluteDerivativeFromConnectionIndexer[v_,Γ_,time_,coords_]:=
976   With[{
977     inertialTerms=CurvatureForcesFromConnectionIndexer[v,Γ,time,coords],
978     coordinateAcceleration=(c→c'[time])/@coords},
979   inertialTerms+coordinateAcceleration];

982 PotentialScalarFieldFromMetricIndexer[V_,invMetric_,time_,coords_]:=
983   With[{injectTime=Flatten[(c→{c→c[time],c'→c'[time]})/@coords]},
984   Table[
985     -ContractIndexedAll[
986       {v}→invMetric[μ,v]*D[V,v],
987       coords],
988   {μ,coords}]/.injectTime];

991 CurvatureForcesUD[
992   gDD_Symbol, gDDRules_,
993   gUU_Symbol, gUURules_,
994   time_Symbol,
995   coords_]:=
996   With[{injectTime=Flatten[(c→{c→c[time],c'→c'[time]})/@coords]},
997   Table[
998     ContractIndexedAll[
999       {v,λ}→(CalculateGammaComponent[
1000         μ,v,λ,
1001         gDD, gDDRules,
1002         gUU, gUURules,
1003         coords]/.injectTime)*D[v[time], time] * D[λ[time], time],
1004         coords],
1005     {μ,coords}]]];

1008 AbsoluteDerivativeUD[
1009   gDD_Symbol, gDDRules_,(* Metric symbol and rules *)
1010   gUU_Symbol, gUURules_, (* Inverse Metric symbol and rules *)
1011   time_Symbol,          (* The time variable, e.g. t *)
1012   coords_               (* The list of coordinate symbols *)
1013 ] :=
1014   With[{
1015     inertialTerms=CurvatureForcesUD[gDD,gDDRules,gUU,gUURules,time,coords],
1016     coordinateAcceleration=(c→c'[time])/@coords},
1017   inertialTerms+coordinateAcceleration];

```

```
1020 ClearAll[PotentialScalarFieldUD];
1021 PotentialScalarFieldUD[V_,gUU_Symbol,gUURules_,time_Symbol,coords_]:=
1022     With[{injectTime=Flatten[(c $\mapsto$ {c $\rightarrow$ c[time],c' $\rightarrow$ c'[time]})/@coords],
1023         GradV=GradRaised[V,gUU,gUURules,coords]},
1024         -GradV/.injectTime
1025     ];
```